ASSESSMENT REPORT

HCLTech Assessment – 2

Model Question Paper – Cloud Data Engineer – Jain University

Total Marks: 100 | Time Limit: 3 Hours

Submitted by

Juved MahammadRafik Attar

USN	2JI23CS056
Branch	Computer Science and Engineering
College	Jain College of Engineering, Belagavi

SECTION A: ADVANCED SQL | Maximum Marks: 20

Question 1: Database Design and Advanced Queries (8 Marks)

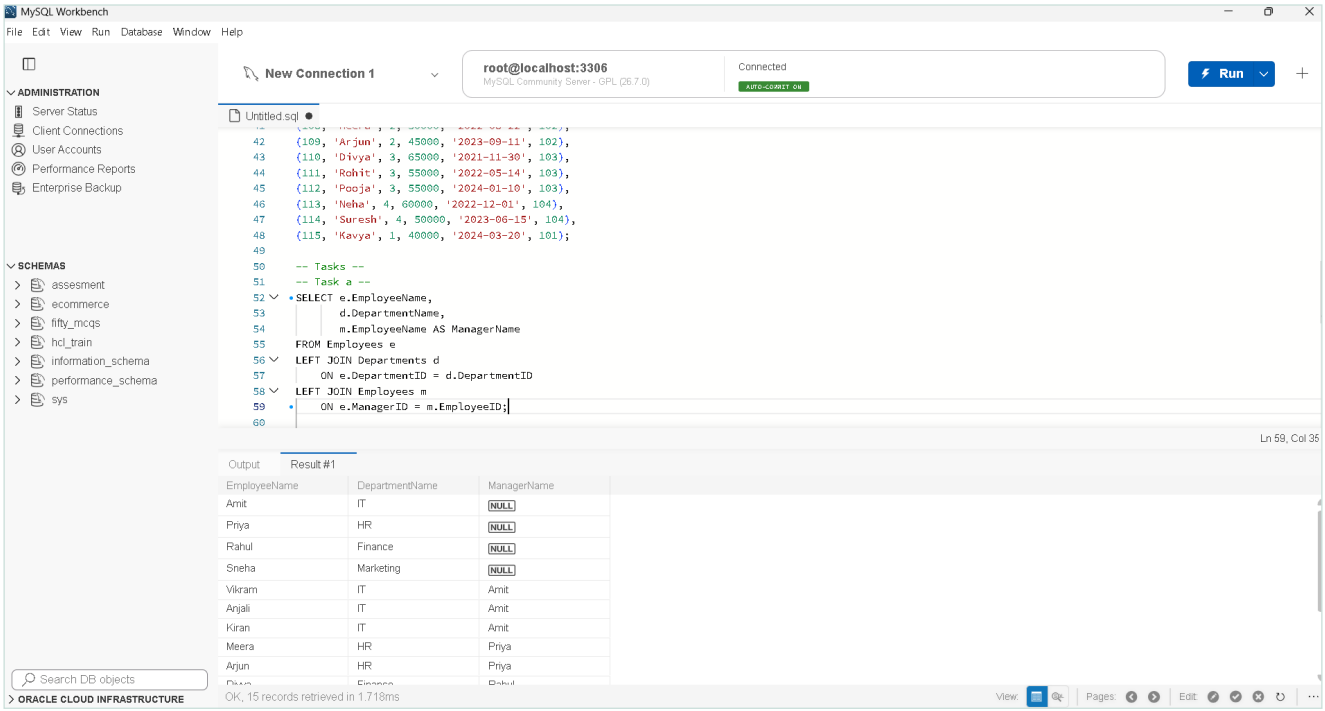
Database and table definitions (reference; the INSERT statements are visible in the screenshots below):

```
CREATE DATABASE EnterpriseDB;
USE EnterpriseDB;

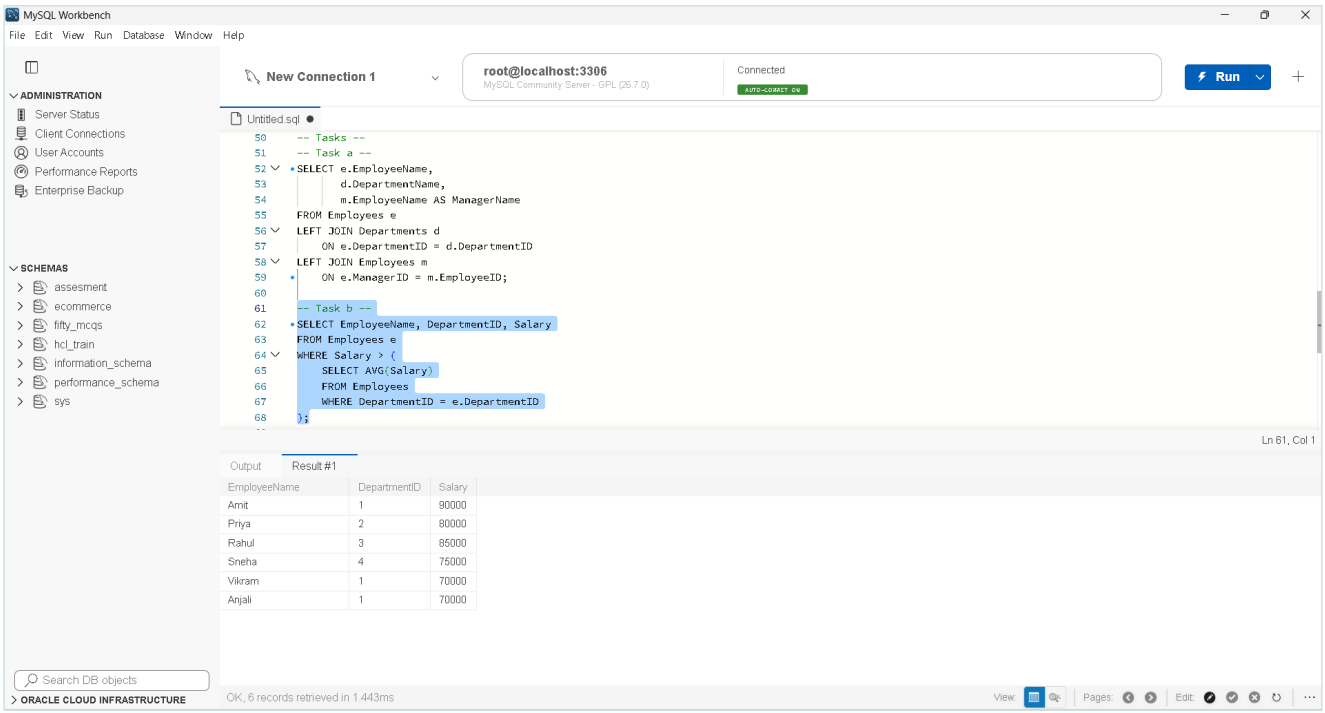
CREATE TABLE Departments (
    DepartmentID   INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL,
    Location       VARCHAR(50)
);

CREATE TABLE Employees (
    EmployeeID   INT PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL,
    DepartmentID INT,
    Salary       INT NOT NULL,
    JoiningDate  DATE,
    ManagerID    INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments (DepartmentID)
);
```

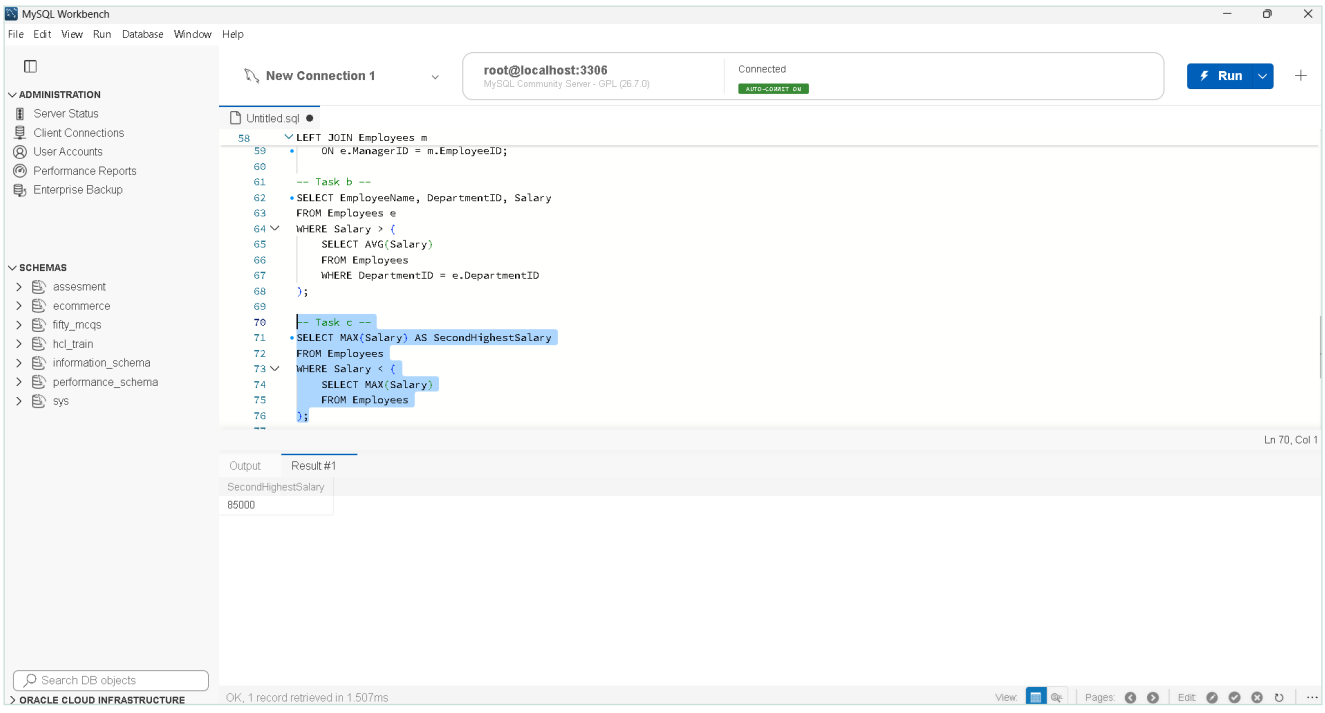
a. Display employee names, department names, and manager names using joins. (2 marks)



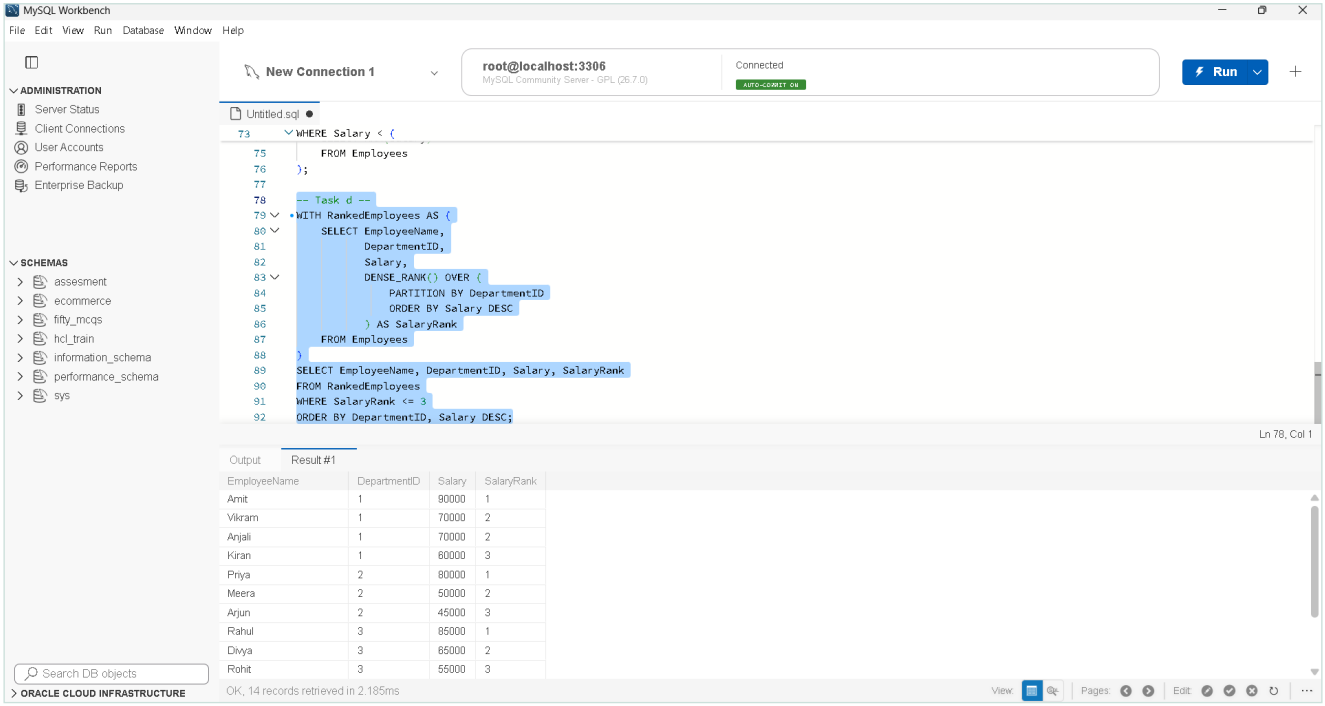
b. Find employees earning more than their department's average salary. (2 marks)



c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH FIRST. (2 marks)



d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)



Alternative using RANK() (a tie at the cutoff keeps every tied employee, so a department can return more than three rows):

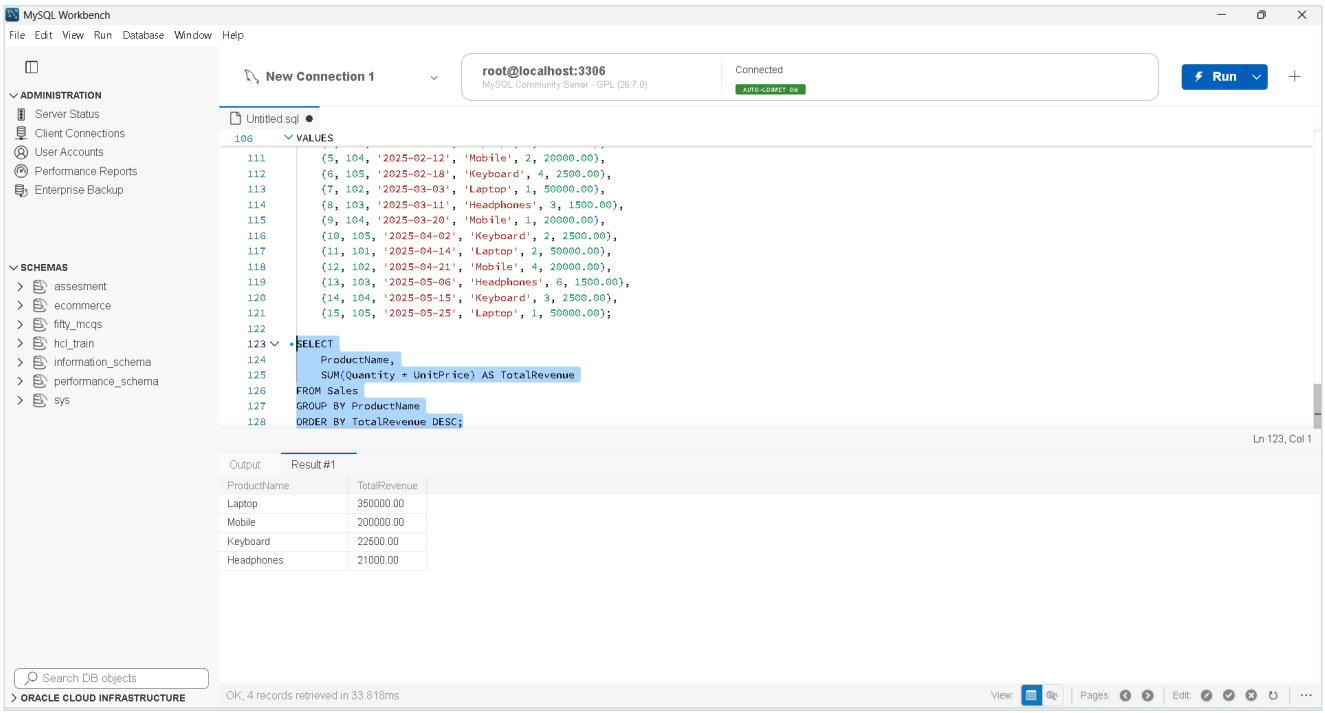
```
WITH RankedEmployees AS (
    SELECT EmployeeName,
           DepartmentID,
           Salary,
           RANK() OVER (
               PARTITION BY DepartmentID
               ORDER BY Salary DESC
           ) AS SalaryRank
    FROM Employees
)
SELECT EmployeeName, DepartmentID, Salary, SalaryRank
FROM RankedEmployees
WHERE SalaryRank <= 3
ORDER BY DepartmentID, Salary DESC;
```

Question 2: Sales Analytics and Query Optimization (6 Marks)

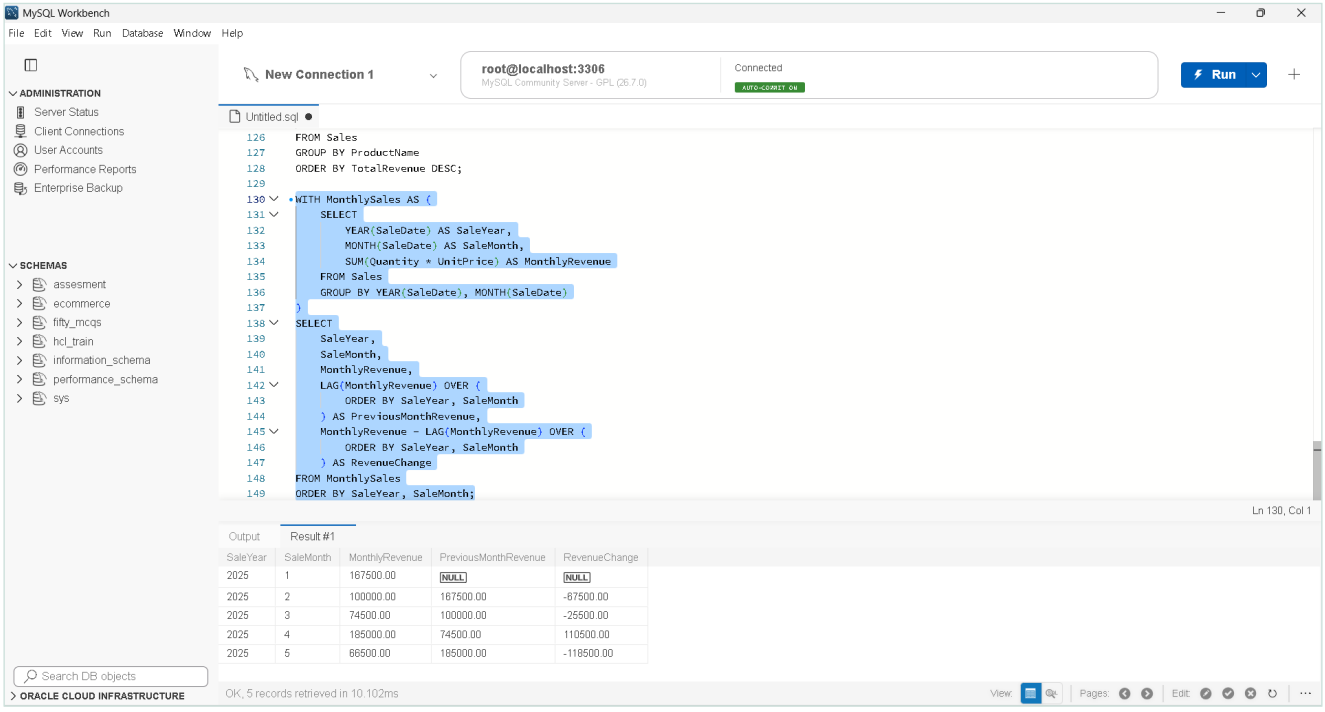
Sales table definition (reference):

```
CREATE TABLE Sales (
    SaleID      INT PRIMARY KEY,
    CustomerID  INT NOT NULL,
    SaleDate    DATE NOT NULL,
    ProductName VARCHAR(100) NOT NULL,
    Quantity    INT NOT NULL,
    UnitPrice   DECIMAL(10,2) NOT NULL
);
```

a. Calculate total revenue for each product. (2 marks)



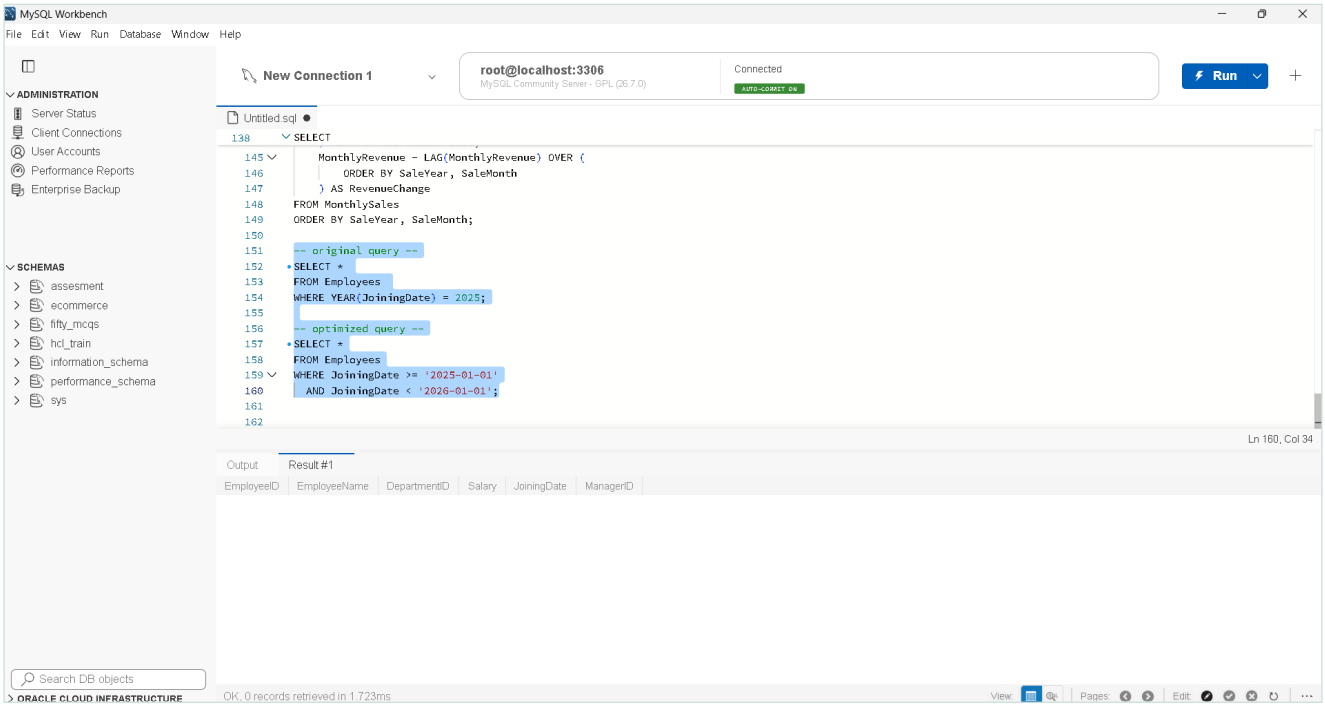
b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)



c. Rewrite the following query to improve the potential for index usage and explain why your version may perform better. (2 marks)

```
SELECT *
FROM Employees
WHERE YEAR(JoiningDate) = 2025;
```

Assume JoiningDate may be indexed.



Answer: Wrapping JoiningDate inside YEAR() forces the database to evaluate the function for every row, so an index on JoiningDate cannot be used and a full table scan is needed. The rewritten query compares the raw column with a date range (on or after 2025-01-01 and before 2026-01-01). The condition can use an index range scan on JoiningDate and read only the matching rows. The half-open range (>= start, < next year start) also remains correct if the column ever stores a time value.

Question 3: SQL Troubleshooting (6 Marks)

a. Explain how an execution plan helps identify expensive operations in a query. (2 marks)

Answer: An execution plan shows the steps the database uses to execute a query. It highlights costly operations such as full table scans, expensive joins, and sorts. It helps identify performance bottlenecks so that indexes or queries can be optimized.

b. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary. Explain your reasoning. (2 marks)

Answer: Create a composite index on DepartmentID and Salary:

```
CREATE INDEX idx_employee_dept_salary
ON Employees (DepartmentID, Salary);
```

Reasoning:

- The index supports queries filtering by both DepartmentID and Salary.
- Placing DepartmentID first is suitable when queries filter by department alone or by department and salary together.

c. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once. (2 marks)

Answer:

```
SELECT EmailID
FROM Employees
GROUP BY EmailID
HAVING COUNT(EmailID) > 1;
```

Equivalent query for a Customers table (Email column):

```
SELECT Email, COUNT(*) AS EmailCount
FROM Customers
GROUP BY Email
HAVING COUNT(*) > 1;
```

SECTION B: LINUX | Maximum Marks: 15

Question 4: Log Analysis (5 Marks)

Assume /var/log/application.log contains millions of log entries in the following format:

```
2026-10-28 10:15:23 ERROR payment-service Transaction failed
```

a. Find all lines containing ERROR. (1 mark)

Answer:

```
grep "ERROR" /var/log/application.log
```

b. Display the 20 most recent error entries in the file. (1 mark)

Answer:

```
grep "ERROR" /var/log/application.log | tail -n 20
```

c. Count the total number of error entries. (1 mark)

Answer:

```
grep -c "ERROR" /var/log/application.log
```

d. Extract error entries for a specified date into a separate file. (1 mark)

Answer:

```
grep "^2026-10-28.*ERROR" /var/log/application.log > errors_2026-10-28.log
```

e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)

Answer: Extract the date and the hour from every ERROR line, then group and count them. The first two fields of each log line are the date and the time, so substr($2, 1, 2) keeps only the hour, and sort | uniq -c counts the errors for each date-and-hour combination:

```
grep "ERROR" /var/log/application.log \
  | awk '{print $1, substr($2, 1, 2)}' \
  | sort | uniq -c
```

Question 5: Permissions and System Troubleshooting (5 Marks)

A Linux server hosts an application that processes confidential financial reports.

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

Answer:

```
sudo groupadd finance
touch report.txt
```

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

Answer:

```
sudo chown "$USER":finance report.txt
chmod 640 report.txt
```

c. Identify the processes consuming the most CPU. (1 mark)

Answer:

```
top
```

d. Display memory usage and filesystem disk utilization. (1 mark)

Answer:

```
free -h
df -h
```

e. Find files larger than 100 MB under a specified application directory. (1 mark)

Answer:

```
find /path/to/application -type f -size +100M
```

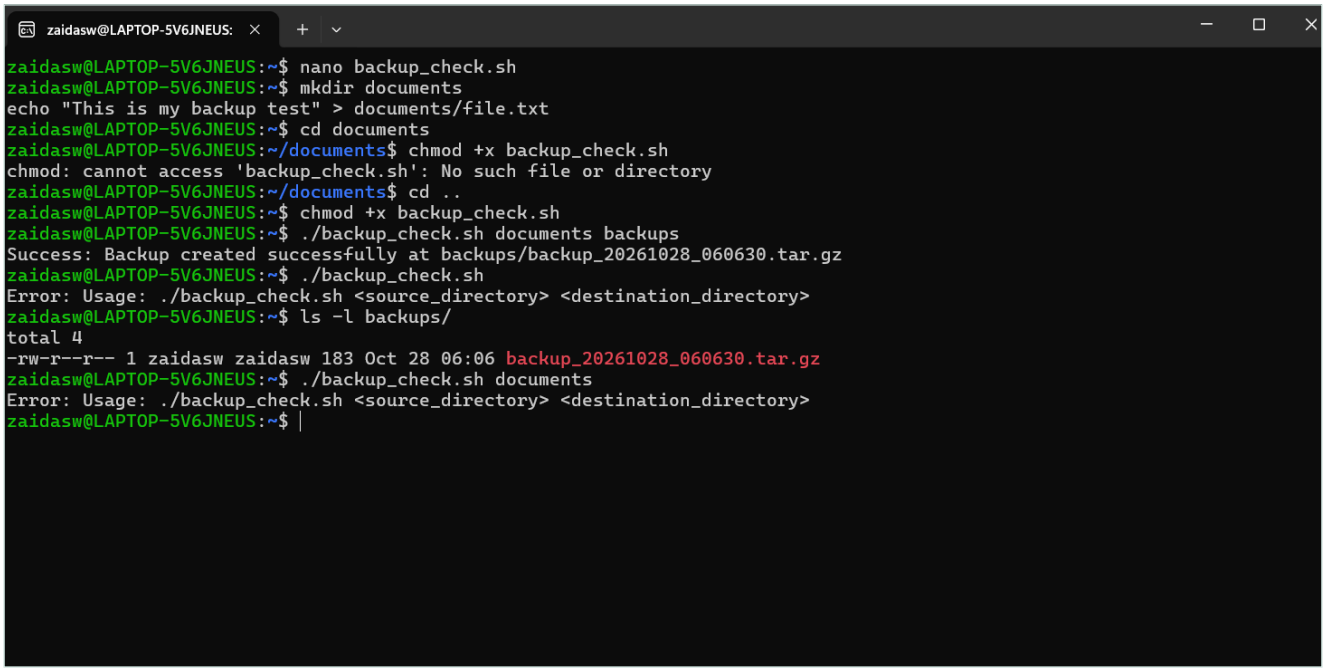
Question 6: Bash Scripting (5 Marks)

Write a Bash script named backup_check.sh that accepts source and destination directories as command-line arguments, validates them, and creates a timestamped compressed backup.

Script: backup_check.sh

```
#!/bin/bash
# Check whether both arguments are provided
if [ "$#" -ne 2 ]; then
    echo "Error: Usage: $0 <source_directory> <destination_directory>"
    exit 1
fi
SOURCE="$1"
DESTINATION="$2"
# Check whether the source directory exists
if [ ! -d "$SOURCE" ]; then
    echo "Error: Source directory '$SOURCE' does not exist."
    exit 1
fi
# Create destination directory if it does not exist
if [ ! -d "$DESTINATION" ]; then
    mkdir -p "$DESTINATION"
    if [ $? -ne 0 ]; then
        echo "Error: Could not create destination directory."
        exit 1
    fi
fi
# Generate a timestamp
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
# Create a compressed archive
ARCHIVE="$DESTINATION/backup_$TIMESTAMP.tar.gz"
if tar -czf "$ARCHIVE" -C "$(dirname "$SOURCE")" "$(basename "$SOURCE")"; then
    echo "Success: Backup created successfully at $ARCHIVE"
else
    echo "Error: Backup failed."
    exit 1
fi
exit 0
```

Execution (valid and invalid arguments):



SECTION C: PYSPARK DATA ENGINEERING | Maximum Marks: 20

Question 7: PySpark DataFrame Operations (6 Marks)

PySpark program:

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("Question7") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Amit", "IT", 75000, "2022-01-10"),
    (2, "Priya", "HR", 45000, "2021-03-15"),
    (3, "Rahul", "IT", 85000, "2020-06-20"),
    (4, "Sneha", "Finance", 65000, "2023-02-01"),
    (5, "Kiran", "HR", 55000, "2022-08-12"),
    (6, "Neha", "IT", 92000, "2019-11-05"),
    (7, "Arjun", "Finance", 70000, "2021-07-18"),
    (8, "Divya", "IT", 60000, "2023-04-10")
]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

print("a. Schema:")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

print("b. Employees earning more than 60000:")
df.filter(col("Salary") > 60000).show()

print("c. Annual Salary:")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.select("EmployeeName", "Salary", "AnnualSalary").show()

print("d. Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

print("e. Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5)
```

Execution results:

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0.
Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
a. Schema:
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|1         |Amit        |IT        |75000 |2022-01-10 |
|2         |Priya       |HR        |45000 |2021-03-15 |
|3         |Rahul       |IT        |85000 |2020-06-20 |
|4         |Sneha       |Finance   |65000 |2023-02-01 |
|5         |Kiran       |HR        |55000 |2022-08-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
b. Employees earning more than 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 75000| 2022-01-10|
|         3|       Rahul|        IT| 85000| 2020-06-20|
|         4|       Sneha|   Finance| 65000| 2023-02-01|
|         6|        Neha|        IT| 92000| 2019-11-05|
|         7|       Arjun|   Finance| 70000| 2021-07-18|
+----------+------------+----------+------+-----------+

c. Annual Salary:
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|        Amit| 75000|      900000|
|       Priya| 45000|      540000|
|       Rahul| 85000|     1020000|
|       Sneha| 65000|      780000|
|       Kiran| 55000|      660000|
|        Neha| 92000|     1104000|
|       Arjun| 70000|      840000|
|       Divya| 60000|      720000|
+------------+------+------------+

d. Department-wise average salary and employee count:
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        IT|      78000.0|            4|
|        HR|      50000.0|            2|
|   Finance|      67500.0|            2|
+----------+-------------+-------------+

e. Five highest-paid employees:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|         6|        Neha|        IT| 92000| 2019-11-05|     1104000|
|         3|       Rahul|        IT| 85000| 2020-06-20|     1020000|
|         1|        Amit|        IT| 75000| 2022-01-10|      900000|
|         7|       Arjun|   Finance| 70000| 2021-07-18|      840000|
|         4|       Sneha|   Finance| 65000| 2023-02-01|      780000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Extended program with 15 records (the question asks for at least 15 sample records)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName("Question7") \
    .master("local[*]") \
    .getOrCreate()

data = [
    (1, "Amit", "IT", 75000, "2022-01-10"),
    (2, "Priya", "HR", 45000, "2021-03-15"),
    (3, "Rahul", "IT", 85000, "2020-06-20"),
    (4, "Sneha", "Finance", 65000, "2023-02-01"),
    (5, "Kiran", "HR", 55000, "2022-08-12"),
    (6, "Neha", "IT", 92000, "2019-11-05"),
    (7, "Arjun", "Finance", 70000, "2021-07-18"),
    (8, "Divya", "IT", 60000, "2023-04-10"),
    (9, "Rohit", "IT", 58000, "2022-05-14"),
    (10, "Anjali", "HR", 48000, "2023-06-15"),
    (11, "Meera", "Finance", 72000, "2021-11-30"),
    (12, "Suresh", "IT", 95000, "2020-03-12"),
    (13, "Pooja", "HR", 52000, "2024-01-10"),
    (14, "Karthik", "Finance", 61000, "2022-09-25"),
    (15, "Kavya", "IT", 66000, "2023-03-20")
]

columns = ["EmployeeID", "EmployeeName", "Department", "Salary", "JoiningDate"]

df = spark.createDataFrame(data, columns)
df = df.withColumn("JoiningDate", col("JoiningDate").cast("date"))

print("a. Schema:")
df.printSchema()

print("First five records:")
df.show(5, truncate=False)

print("b. Employees earning more than 60000:")
df.filter(col("Salary") > 60000).show()

print("c. Annual Salary:")
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.select("EmployeeName", "Salary", "AnnualSalary").show()

print("d. Department-wise average salary and employee count:")
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()

print("e. Five highest-paid employees:")
df.orderBy(col("Salary").desc()).show(5)
```

Expected output (calculated from the 15 records above; not an execution screenshot):

```
a. Schema:
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

First five records:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|1         |Amit        |IT        |75000 |2022-01-10 |
|2         |Priya       |HR        |45000 |2021-03-15 |
|3         |Rahul       |IT        |85000 |2020-06-20 |
|4         |Sneha       |Finance   |65000 |2023-02-01 |
|5         |Kiran       |HR        |55000 |2022-08-12 |
+----------+------------+----------+------+-----------+
only showing top 5 rows
```

```
b. Employees earning more than 60000:
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|        Amit|        IT| 75000| 2022-01-10|
|         3|       Rahul|        IT| 85000| 2020-06-20|
|         4|       Sneha|   Finance| 65000| 2023-02-01|
|         6|        Neha|        IT| 92000| 2019-11-05|
|         7|       Arjun|   Finance| 70000| 2021-07-18|
|        11|       Meera|   Finance| 72000| 2021-11-30|
|        12|      Suresh|        IT| 95000| 2020-03-12|
|        14|     Karthik|   Finance| 61000| 2022-09-25|
|        15|       Kavya|        IT| 66000| 2023-03-20|
+----------+------------+----------+------+-----------+
```

```
c. Annual Salary:
+------------+------+------------+
|EmployeeName|Salary|AnnualSalary|
+------------+------+------------+
|        Amit| 75000|      900000|
|       Priya| 45000|      540000|
|       Rahul| 85000|     1020000|
|       Sneha| 65000|      780000|
|       Kiran| 55000|      660000|
|        Neha| 92000|     1104000|
|       Arjun| 70000|      840000|
|       Divya| 60000|      720000|
|       Rohit| 58000|      696000|
|      Anjali| 48000|      576000|
|       Meera| 72000|      864000|
|      Suresh| 95000|     1140000|
|       Pooja| 52000|      624000|
|     Karthik| 61000|      732000|
|       Kavya| 66000|      792000|
+------------+------+------------+
```

```
d. Department-wise average salary and employee count:
+----------+-----------------+-------------+
|Department|    AverageSalary|EmployeeCount|
+----------+-----------------+-------------+
|        IT|75857.14285714286|            7|
|        HR|          50000.0|            4|
|   Finance|          67000.0|            4|
+----------+-----------------+-------------+
```

```
e. Five highest-paid employees:
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|        12|      Suresh|        IT| 95000| 2020-03-12|     1140000|
|         6|        Neha|        IT| 92000| 2019-11-05|     1104000|
|         3|       Rahul|        IT| 85000| 2020-06-20|     1020000|
|         1|        Amit|        IT| 75000| 2022-01-10|      900000|
|        11|       Meera|   Finance| 72000| 2021-11-30|      864000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

Question 8: Data Cleaning and Aggregation (7 Marks)

PySpark program:

```
[4]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date, sum
from pyspark.sql.functions import expr


spark = SparkSession.builder.appName("DataCleaning").getOrCreate()

df = spark.read.csv("transactions.csv", header=True, inferSchema=True)

print("Original Data:")
df.show()

# Remove duplicates
df = df.dropDuplicates(["TransactionID"])


df = df.withColumn(
    "TransactionDate",
    expr("try_to_date(TransactionDate, 'yyyy-MM-dd')")
)
```

```
df = df.filter(
    col("CustomerID").isNotNull() &
    col("TransactionDate").isNotNull() &
    col("Quantity").isNotNull() &
    col("UnitPrice").isNotNull()
)

df = df.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

print("Cleaned Data:")
df.show()

print("Revenue by Product:")
df.groupBy("Product").agg(
    sum("Revenue").alias("TotalRevenue")
).orderBy(col("TotalRevenue").desc()).show()
```

Execution results (original data, cleaned data and revenue by product):

```
Original Data:
+-------------------+----------+---------------+--------+--------+---------+
|      TransactionID|CustomerID|TransactionDate| Product|Quantity|UnitPrice|
+-------------------+----------+---------------+--------+--------+---------+
|2026-10-28 01:00:00|        C1|     2026-01-10|  Laptop|       1|    50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 02:00:00|        C2|     2026-01-11|   Mouse|       2|      500|
|2026-10-28 03:00:00|        C3|        invalid|Keyboard|       1|     1500|
|2026-10-28 04:00:00|      NULL|     2026-01-12|  Laptop|       1|    50000|
+-------------------+----------+---------------+--------+--------+---------+

Cleaned Data:
+-------------------+----------+---------------+-------+--------+---------+-------+
|      TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------------+----------+---------------+-------+--------+---------+-------+
|2026-10-28 01:00:00|        C1|     2026-01-10| Laptop|       1|    50000|  50000|
|2026-10-28 02:00:00|        C2|     2026-01-11|  Mouse|       2|      500|   1000|
+-------------------+----------+---------------+-------+--------+---------+-------+

Revenue by Product:
+-------+------------+
|Product|TotalRevenue|
+-------+------------+
| Laptop|       50000|
|  Mouse|        1000|
+-------+------------+
```

c. Handle missing values in required fields and explain your chosen approach. (2 marks)

Answer: CustomerID, TransactionDate, Quantity and UnitPrice are required fields. A missing customer, date, quantity or price cannot be guessed reliably, and filling them with default values would produce wrong revenue figures. The program therefore removes the rows where any of these fields is NULL using filter(isNotNull()). Invalid dates such as "invalid" are converted to NULL by try_to_date(), so the same filter also removes them. If a non-critical field (for example Product) were missing, it could be replaced with a default value such as "Unknown" using fillna() instead of dropping the row.

Question 9: PySpark Joins and Performance (7 Marks)

a. Load both datasets into DataFrames. (1 mark)

[1]:
```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, broadcast

spark = SparkSession.builder \
    .appName("CustomerOrders") \
    .getOrCreate()

customers_data = [
    (1, "Amit", "Bengaluru"),
    (2, "Priya", "Mumbai"),
    (3, "Rahul", "Delhi"),
    (4, "Sneha", "Bengaluru"),
    (5, "Kiran", "Chennai")
]

orders_data = [
    (101, 1, 5000.0, "2026-01-10"),
    (102, 2, 3000.0, "2026-01-12"),
    (103, 1, 2000.0, "2026-01-15"),
    (104, 3, 7000.0, "2026-01-18"),
    (105, 4, 4000.0, "2026-01-20")
]

customers = spark.createDataFrame(
    customers_data,
    ["CustomerID", "CustomerName", "City"]
)

orders = spark.createDataFrame(
    orders_data,
    ["OrderID", "CustomerID", "OrderAmount", "OrderDate"]
)

print("Customers:")
customers.show()

print("Orders:")
orders.show()
```

```
C:\Users\Anmol\dataEngineer\.venv\Lib\site-packages\pyspark\testing
0. Some features may not work correctly. It is recommended to use p
  require_minimum_pandas_version()
Customers:
+----------+------------+---------+
|CustomerID|CustomerName|     City|
+----------+------------+---------+
|         1|        Amit|Bengaluru|
|         2|       Priya|   Mumbai|
|         3|       Rahul|    Delhi|
|         4|       Sneha|Bengaluru|
|         5|       Kiran|  Chennai|
+----------+------------+---------+

Orders:
+-------+----------+-----------+----------+
|OrderID|CustomerID|OrderAmount| OrderDate|
+-------+----------+-----------+----------+
|    101|         1|     5000.0|2026-01-10|
|    102|         2|     3000.0|2026-01-12|
|    103|         1|     2000.0|2026-01-15|
|    104|         3|     7000.0|2026-01-18|
|    105|         4|     4000.0|2026-01-20|
+-------+----------+-----------+----------+
```

b. Join the datasets to display orders with customer details. (2 marks)

```
joined = orders.join(customers, "CustomerID", "inner")

print("Orders with customer details:")
joined.show()
```

```
Orders with customer details:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

c. Identify customers who have never placed an order. (1 mark)

```
[3]: print("Customers without orders:")

customers.join(
    orders.select("CustomerID").distinct(),
    "CustomerID",
    "left_anti"
).show()
```

```
Customers without orders:
+----------+------------+-------+
|CustomerID|CustomerName|   City|
+----------+------------+-------+
|         5|       Kiran|Chennai|
+----------+------------+-------+
```

d. Calculate the total order value by city. (1 mark)

```
from pyspark.sql.functions import col, sum

orders.join(customers, "CustomerID", "inner") \
    .groupBy("City") \
    .agg(sum("OrderAmount").alias("TotalOrderValue")) \
    .orderBy(col("TotalOrderValue").desc()) \
    .show()
```

Expected output (calculated from the sample data above; not an execution screenshot):

```
Total order value by city:
+---------+---------------+
|     City|TotalOrderValue|
+---------+---------------+
|Bengaluru|        11000.0|
|    Delhi|         7000.0|
|   Mumbai|         3000.0|
+---------+---------------+
```

e. Explain when a broadcast join is useful and what conditions make it appropriate. (1 mark)

```
print("Broadcast join:")

broadcast(orders).join(customers, "CustomerID").show()
```

```
Broadcast join:
+----------+-------+-----------+----------+------------+---------+
|CustomerID|OrderID|OrderAmount| OrderDate|CustomerName|     City|
+----------+-------+-----------+----------+------------+---------+
|         1|    103|     2000.0|2026-01-15|        Amit|Bengaluru|
|         1|    101|     5000.0|2026-01-10|        Amit|Bengaluru|
|         2|    102|     3000.0|2026-01-12|       Priya|   Mumbai|
|         3|    104|     7000.0|2026-01-18|       Rahul|    Delhi|
|         4|    105|     4000.0|2026-01-20|       Sneha|Bengaluru|
+----------+-------+-----------+----------+------------+---------+
```

Broadcast join example

Answer: A broadcast join sends a full copy of a small DataFrame to every executor, so Spark can join it with the large DataFrame locally without shuffling the large dataset across the network. This makes the join much faster. It is appropriate when:

- One side of the join is small enough to fit in the memory of each executor (by default below spark.sql.autoBroadcastJoinThreshold, 10 MB, or when forced with broadcast()).
- The other side is large, so avoiding its shuffle gives a real saving.
- The join is an equality join, such as joining on CustomerID.

It is not suitable when both datasets are large, because broadcasting a big table can cause out-of-memory errors. In this program the small table is broadcast with broadcast(orders).join(customers, "CustomerID").

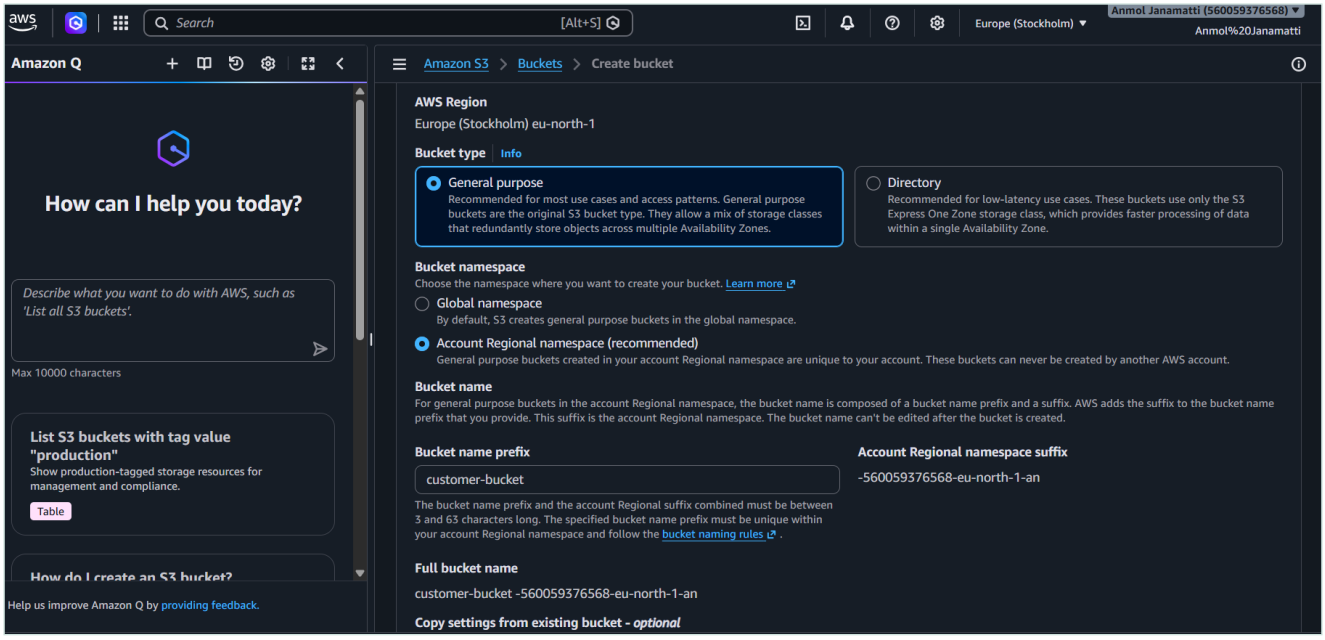
f. Explain the difference between transformations and actions, giving one example of each. (1 mark)

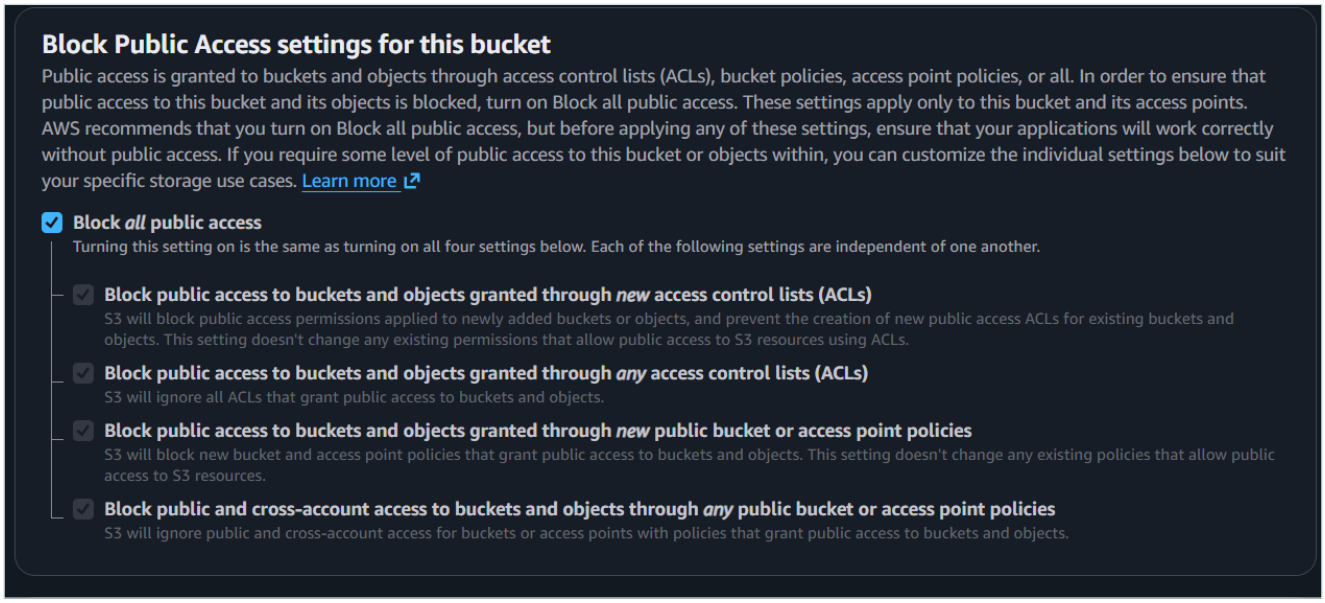
- Transformations: Create a new DataFrame or execution plan. Example: filter().
- Actions: Execute computations and return results. Example: show().

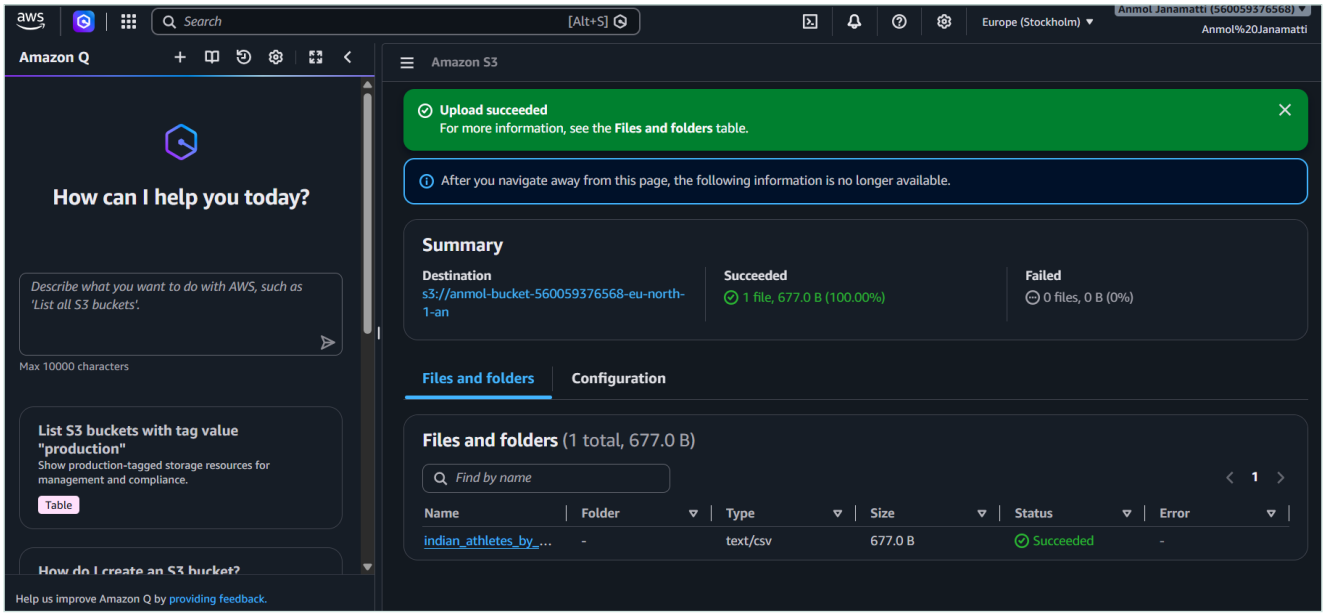
SECTION D: AWS CLOUD SERVICES | Maximum Marks: 15

Question 10: S3 and IAM Security (5 Marks)

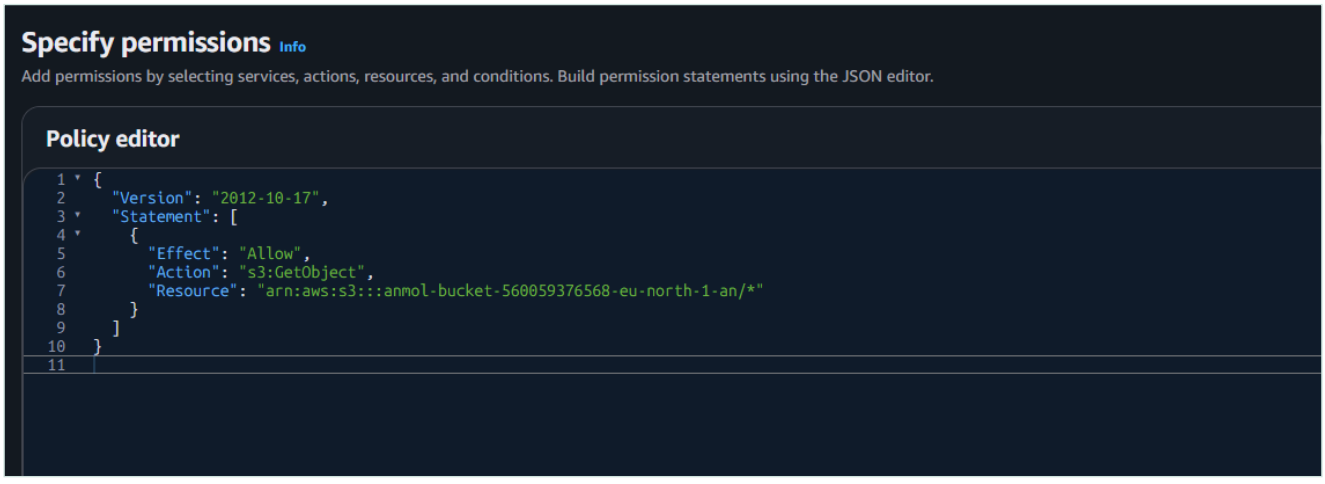
a. Create an S3 bucket with Block Public Access enabled and upload a sample file. (1 mark)







b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)



c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)

Answer: If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data. (1 mark)

Answer: Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

Question 11: EC2 Troubleshooting and CloudWatch (5 Marks)

A web application hosted on EC2 becomes slow, and users report connectivity problems.

a. Check the EC2 instance state and status checks. (1 mark)

Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity. (1 mark)

Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period. (2 marks)

Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization. (1 mark)

Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

Question 12: Event-Driven Processing (5 Marks)

A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.

a. Identify a suitable service for executing the processing code. (1 mark)

AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

b. Explain how an S3 upload can trigger processing. (1 mark)

Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)

Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

d. Explain how errors and processing logs can be captured. (1 mark)

Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

e. Explain how to prevent duplicate processing or recursive triggering. (1 mark)

Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

SECTION E: AWS STEP FUNCTIONS | Maximum Marks: 15

Question 13: State Machine Design (7 Marks)

A company needs to process sales files through these stages: validate the input file, transform the data, calculate total sales by product, store the results in Amazon S3, and send a completion notification. Design an AWS Step Functions state machine to coordinate the workflow.

a. Identify suitable states for validation, processing, storage, and notification. (2 marks)

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)

Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

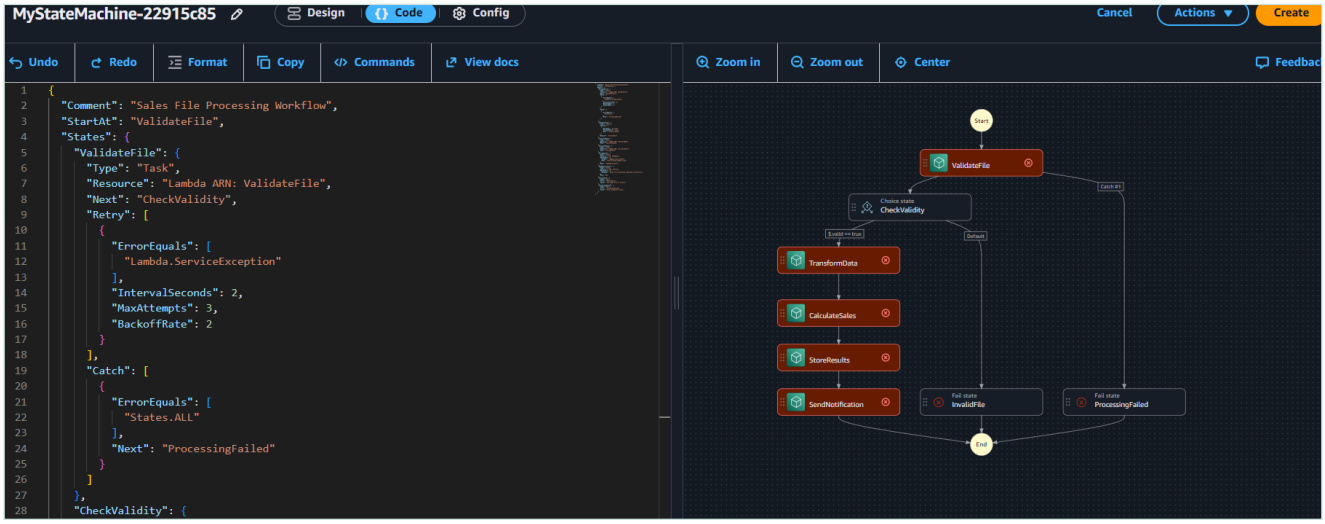
d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.

Expected output – workflow definition and diagram:



Question 14: Step Functions Implementation (8 Marks)

A data engineering team has developed three processing functions: ValidateFile, TransformData, and LoadResults. The functions must execute in sequence.

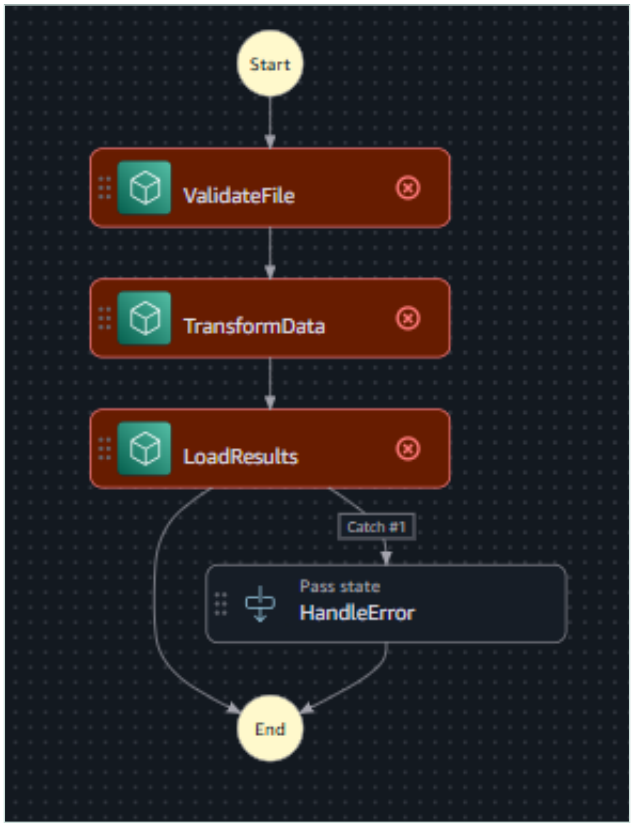
Requirements:

- If validation fails, the workflow must stop.
- If transformation fails temporarily, the workflow must retry.
- If loading fails permanently, the workflow must record the failure.

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary. (3 marks)

```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": [
            "Lambda.ServiceException"
          ],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
```

```
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": [
            "States.ALL"
          ],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```



Workflow graph

ASL definition (text version of the screenshots above):

```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": ["Lambda.ServiceException"],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts. (2 marks)

The Retry block retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.

c. Configure a Catch transition for the loading task that routes to an error-handling state. (1 mark)

The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.

d. Explain how data from one task is passed to the next task. (1 mark)

Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.

e. Explain how you would test successful and failed executions. (1 mark)

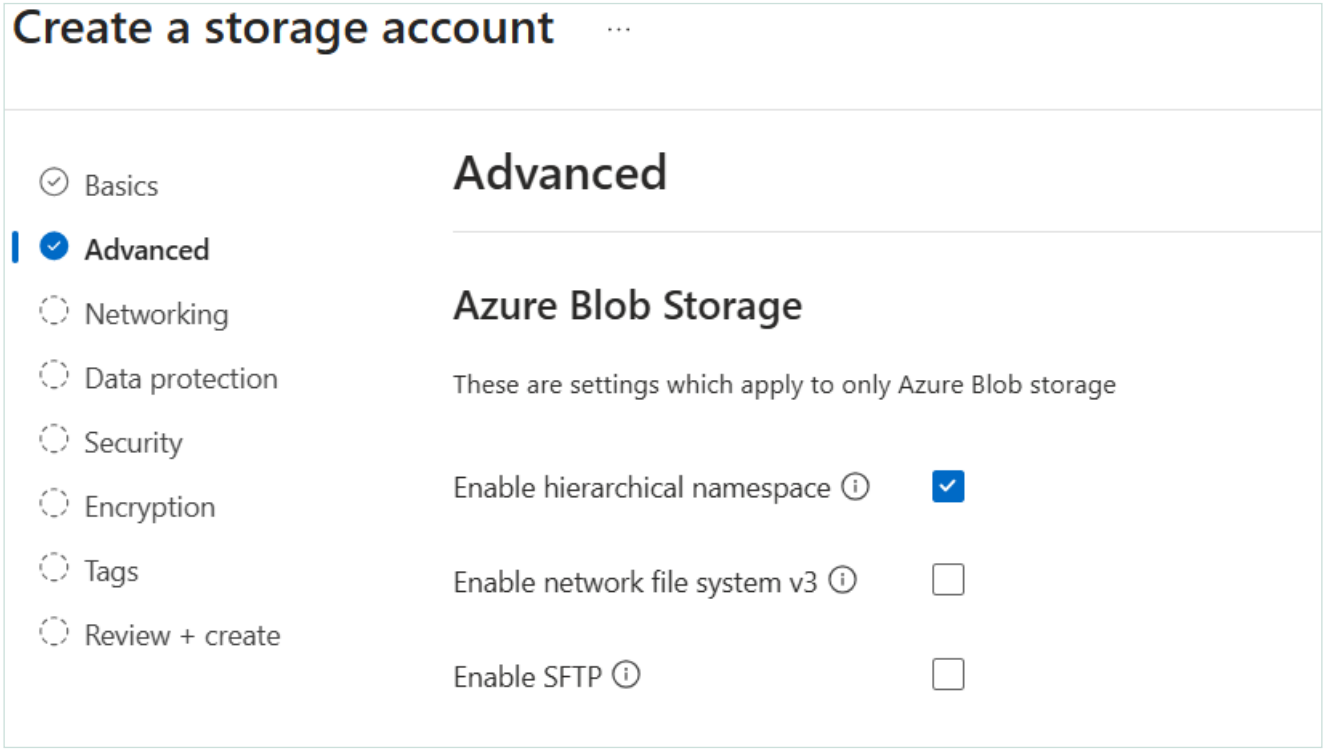
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

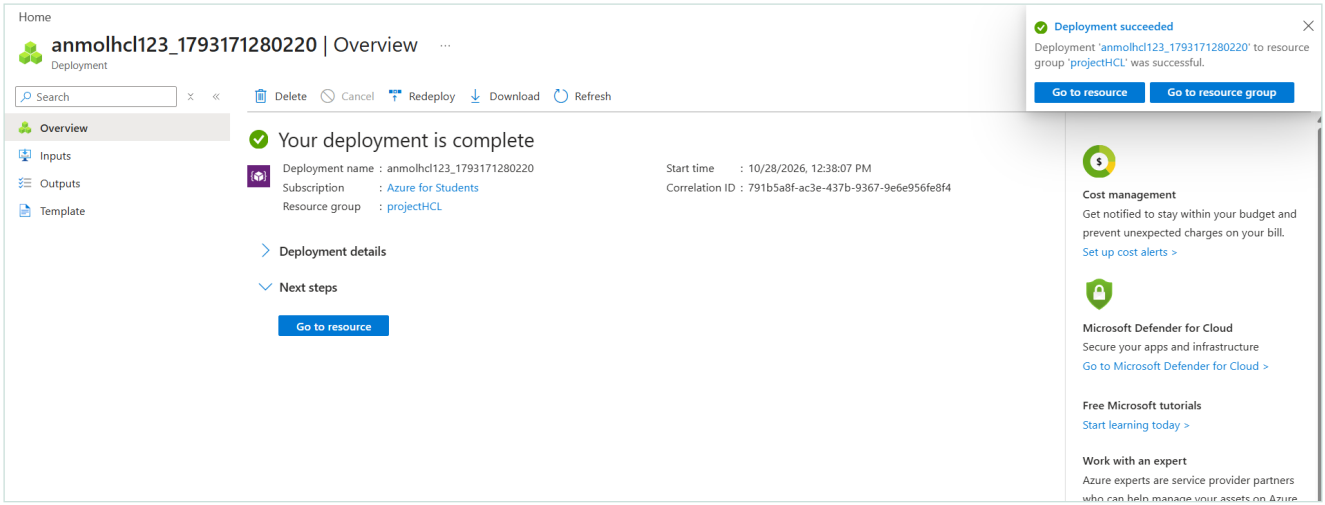
SECTION F: MICROSOFT AZURE | Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks)

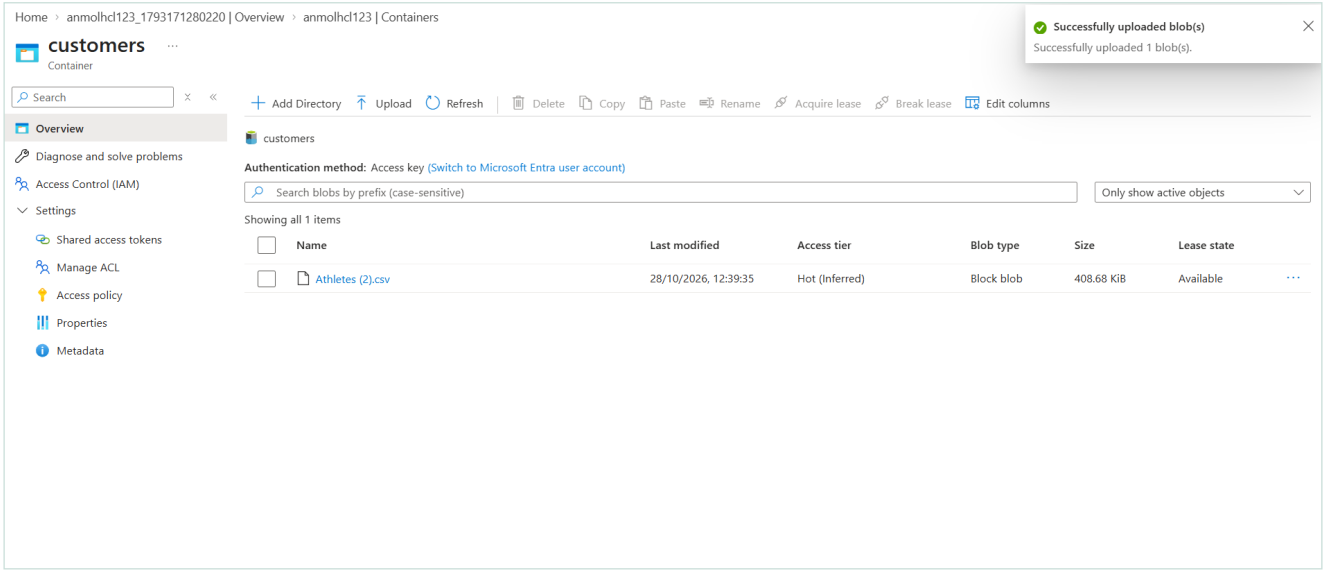
a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)

Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.





b. Create a filesystem or container and upload a sample dataset. (1 mark)



Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured.

Question 16: Azure VM Troubleshooting (5 Marks)

A virtual machine is running in Azure, but administrators cannot connect to it.

a. Verify the VM's provisioning and power states. (1 mark)

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how Network Security Group rules can prevent connectivity. (1 mark)

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Explain how subnet and routing configuration affect connectivity. (1 mark)

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet. (2 marks)

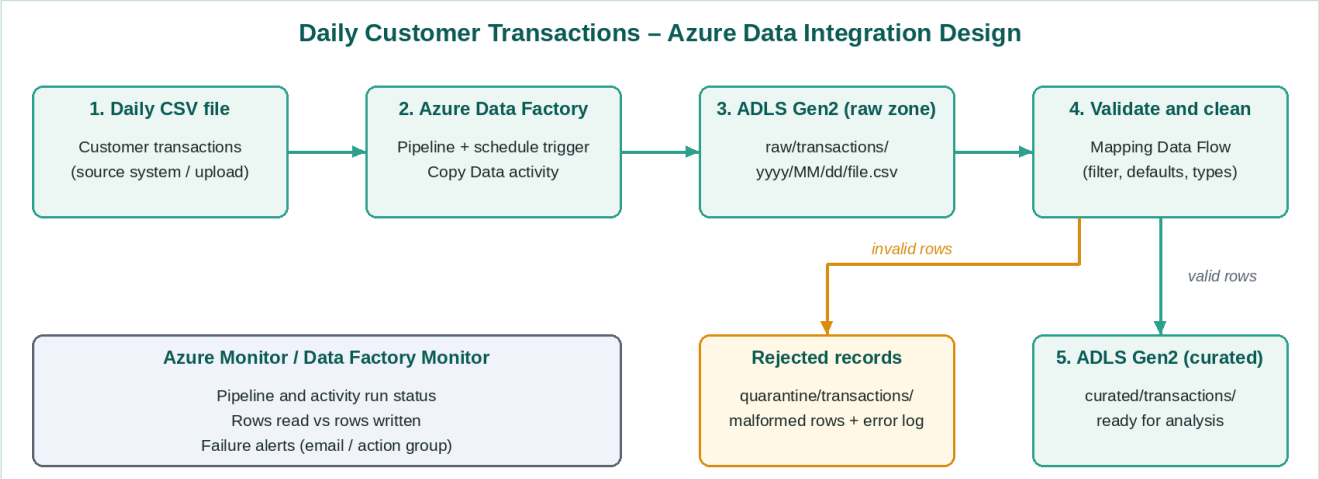
Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.

Question 17: Azure Data Integration (5 Marks)

A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.

a. Select an appropriate service to orchestrate data movement. (1 mark)

Answer: Azure Data Factory (ADF) is suitable for orchestrating the data movement. It provides pipelines, activities, linked services and triggers to move and transform data on a schedule.



Solution design

b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)

Answer: Create linked services for the source and for the ADLS Gen2 storage account. Create a DelimitedText dataset for the CSV file and a sink dataset for the ADLS Gen2 container. Add a Copy Data activity to a pipeline that reads the CSV from the source and writes it to the raw folder of the ADLS Gen2 filesystem. Run the pipeline daily with a schedule trigger.

c. Describe how to organize files by ingestion date. (1 mark)

Answer: Use dynamic content in the sink dataset path so that each daily file is stored in a folder named after the ingestion date, for example:

```
raw/transactions/@{formatDateTime(utcNow(), 'yyyy/MM/dd')}/transactions.csv
```

This creates folders such as raw/transactions/2026/10/28/, which keeps the data organized, makes it easy to reprocess a specific day, and supports partition pruning during analysis.

d. Explain how to handle malformed records and missing values. (1 mark)

Answer: In the Copy Data activity, enable fault tolerance to skip incompatible rows and log them to a separate quarantine folder. In a Mapping Data Flow, validate the data with a Filter or Conditional Split transformation: rows with valid required fields continue to the curated zone, and malformed rows go to the quarantine folder. Missing values in non-critical columns can be replaced with defaults using a Derived Column transformation (for example, coalesce()), while rows with missing required values such as a transaction ID or amount are rejected.

e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)

Answer: Use the Monitor tab in Azure Data Factory to track pipeline and activity runs and to view failures. Configure Azure Monitor alerts to notify the team when a pipeline fails. To validate completeness, compare the rows read and rows written shown in the Copy Data activity output, check the number of skipped rows, and confirm that the expected file for the day exists in the ADLS Gen2 folder.